

Lab 1: Première ressource AWS

Durée: 20 minutes

- Tâche 1: Générez votre première configuration Terraform
- Tâche 2: utiliser la CLI Terraform pour obtenir de l'aide
- Tâche 3: Appliquer et mettre à jour votre configuration

Tâche 1: Générez votre première configuration Terraform

Le code Terraform est écrit dans un langage appelé **HCL** dans des fichiers avec l'extension **.tf**. Il s'agit d'un langage déclaratif, qui est également utilisé par les langages de configuration dans d'autres applications, et en particulier par d'autres produits HashiCorp (créateur de Terraform). Notre objectif est donc de décrire l'infrastructure qu'on souhaite dans le langage HCL, et ensuite Terraform s'occupe de comment la créer.

Remarque : L'éditeur Visual code dispose d'un plugin Terraform pour valider le code écrit (<https://marketplace.visualstudio.com/items?itemName=mauve.terraform>)

Étape 1.1

Créer un répertoire "terraform" pour contenir les fichiers du projet.

Dans le répertoire "terraform", créez le fichier intitulé `main.tf` pour créer une instance AWS avec les propriétés suivantes:

- ami
- subnet_id
- vpc_security_group_ids
- tags.Identity

Création du Provider (fournisseur)

La première étape de l'utilisation de Terraform consiste généralement à configurer le ou les providers (fournisseurs) que vous souhaitez utiliser. Pour ce faire, créez un fichier avec l'extension **.tf**, dans mon cas je vais créer un fichier nommé `main.tf`. Ensuite, mettez-y le code suivant:

```
provider "aws" {  
  access_key = "<YOU_ACCESS_KEY>"  
  secret_key = "<YOUR_SECRET_KEY>"  
  region     = "eu-west-1"  
}
```

Cela indique à Terraform que vous allez utiliser le fournisseur [AWS](#) et que vous souhaitez déployer votre infrastructure dans la région `eu-west-1`. Vous devez également spécifier la paire de clés de votre utilisateur générée auparavant.

Création de la Resource (ressource)

Pour chaque fournisseur, il existe de nombreux types de ressources que vous pouvez créer, tels que des serveurs, des bases de données, des équilibrateurs de charge, etc.

Avant de déployer des ressources complexes, voyons d'abord comment déployer une instance de calcul (machine virtuelle). Dans le jargon AWS, un serveur/machine virtuel est appelé une instance EC2 .

Ajoutez le code suivant à votre fichier main.tf :

```
resource "aws_instance" "web" {  
  ami          = "ami-0f29c8402f8cce65c"  
  instance_type = "t2.micro"  
}
```

La syntaxe générale d'une ressource Terraform est la suivante :

```
resource "<PROVIDER>_<TYPE>" "<RESOURCE_NAME>" {  
  [CONFIG ...]  
}
```

- **PROVIDER** : c'est le nom d'un fournisseur (ici le provider "aws").
- **TYPE** : c'est le type de ressources à créer dans ce fournisseur (ici c'est une instance ec2)
- **RESOURCE_NAME** : c'est un identifiant que vous pouvez utiliser dans le code Terraform pour faire référence à cette ressource (ici "my_ec2_instance")
- **CONFIG** : se compose de un ou plusieurs arguments spécifiques à cette ressource, dans notre cas :
- **ami** : c'est l'acronyme d'**Amazon Machine Image** (AMI) , c'est donc l'image qui sera exécutée sur notre instance EC2. Vous pouvez trouver des AMI gratuites et payantes sur AWS Marketplace ou créer les vôtres directement depuis la console AWS. Dans notre cas, nous utilisons l'identifiant "ami-0f29c8402f8cce65c" qui est une AMI Ubuntu 16.04 (Attention l'identifiant peut être modifié avec le temps !). Cette AMI est gratuite et éligible à l'offre gratuite d'AWS.
- **instance_type** : Type d'instance EC2 ou **gabarit** à exécuter. Chaque type d'instance EC2 fournit une quantité différente de CPU, de mémoire, d'espace disque et de capacité réseau. Dans notre cas, nous utilisons le type **t2.micro**, qui fait partie du niveau gratuit AWS, et qui a comme caractéristiques 1 vCPU, ainsi que 1 Go de mémoire.

Votre fichier final `main.tf` devrait ressembler à ceci avec des valeurs différentes:

```
provider "aws" {
  access_key = "AKIAIUDVJEEI5KCIKJ6Q"
  secret_key = "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"
  region     = "eu-west-1"
}

resource "aws_instance" "web" {
  ami          = "ami-0f29c8402f8cce65c"
  instance_type = "t2.micro"
}
```

N'oubliez pas de sauvegarder le fichier avant de continuer!

Tâche 4: utiliser la CLI Terraform pour obtenir de l'aide

Étape 4.1

Exécutez la commande suivante pour afficher les commandes disponibles:

```
terraform -help

Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.

Common commands:
  apply          Builds or changes infrastructure
  console        Interactive console for Terraform interpolations
  destroy        Destroy Terraform-managed infrastructure
  env            Workspace management
  fmt            Rewrites config files to canonical format
  get            Download and install modules for the configuration
  graph          Create a visual graph of Terraform resources
  import         Import existing infrastructure into Terraform
  init           Initialize a Terraform working directory
  output         Read an output from a state file
  plan           Generate and show an execution plan

...
```

- (sortie complète tronquée par souci de brièveté)

Ou, vous pouvez utiliser le raccourci:

```
terraform -h
```

Étape 4.2

Accédez au répertoire Terraform et initialisez le projet

```
cd terraform

terraform init

Initializing provider plugins...
...

Terraform has been successfully initialized!
```

terraform init demande à Terraform de d'abord scanner votre code, qui déterminera quel fournisseur vous utilisez et téléchargera son code.

Le code du fournisseur sera téléchargé dans le dossier **.terraform** qui est le répertoire de travail de Terraform. Vous devez exécuter cette commande **init** chaque fois que vous démarrez avec du nouveau code Terraform.

Étape 4.3

Obtenez de l'aide sur la commande **plan**, puis exécutez-la:

```
terraform -h plan
terraform plan
```

Tâche 5: Appliquer et mettre à jour votre configuration

Étape 5.1

Exécutez la commande **terraform apply** pour générer des ressources réelles dans AWS

```
terraform apply
```

Vous serez invité à confirmer les modifications avant leur application. Répondez avec **yes**.

Étape 5.2

Utilisez la commande **terraform show** pour afficher les ressources créées et trouver l'adresse IP de votre instance. Envoyez un ping à cette adresse pour vous assurer que l'instance est en cours d'exécution.

Étape 5.3

Terraform peut effectuer des mises à jour à chaud sur vos instances une fois que des modifications sont apportées au fichier de configuration **main.tf**.

Ajoutez des informations réseaux et sécurité à l'instance que vous trouvez dans le dashboard AWS:

- subnet_id
- vpc_security_group_ids

```
subnet_id          = "subnet-97c0fdfe"
vpc_security_group_ids = ["sg-050a4ca5631347b37"]
```

Ajoutez la section des tags à l'instance AWS:

- Identity
- Name
- Environment

```
tags = {
  "Identity"    = "terraform"
  "Name"        = "Web"
  "Environment" = "Training"
}
```

Votre fichier final `main.tf` devrait ressembler à ceci avec des valeurs différentes:

```
provider "aws" {
  access_key = "AKIAIUDVJEEI5KCIKJ6Q"
  secret_key = "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"
  region     = "eu-west-1"
}

resource "aws_instance" "web" {
  ami          = "ami-0f29c8402f8cce65c"
  instance_type = "t2.micro"

  subnet_id          = "subnet-97c0fdfe"
  vpc_security_group_ids = ["sg-050a4ca5631347b37"]

  tags = {
    "Identity"    = "terraform"
    "Name"        = "Web"
    "Environment" = "Training"
  }
}
```

Étape 5.4

Planifiez et appliquez les modifications que vous venez d'effectuer et notez les différences de sortie pour les ajouts, les suppressions et les modifications.

```
terraform apply
```

Vous devriez voir une sortie indiquant que `aws_instance.web` sera modifié:

```
...
# aws_instance.web will be updated in-place
~ resource "aws_instance" "web" {
...

```

Lorsque vous êtes invité à appliquer les modifications, répondez par `yes`.